

WEATHER + MAP MINI WEB APP

Software Requirements

Simple visual specification for the dashboard, city detail view, backend, testing, security, and deployment

Version	1.0
Status	Draft for development
Scope	Australia, Singapore, and India

Product idea A responsive web app that shows current weather for supported cities and opens a detailed map view when a city is selected.

1. Product overview

Goal Give learners a small but complete full-stack example that is easy to understand, test, secure, and deploy.

Primary user flow



In scope

- Dashboard grouped by country with national flags, current temperature, and a weather emoji.
- City detail view with a map, condition, minimum/maximum temperature, humidity, and a Back action.
- Node.js backend that validates supported input and calls Azure Maps.
- Unit tests, GitHub Actions, Dependabot, CodeQL, Docker packaging, and Azure App Service deployment.

Out of scope for version 1.0

- User accounts or sign-in.
- A database or caching layer.
- Editing the city list from the UI.
- Payments, notifications, or advanced analytics.

Success criteria

Area	Expected result
Usability	A user can reach city details in one click.
Data	The UI displays weather returned by the backend only.
Security	No Azure Maps key appears in source code or browser requests.
Quality	Pull requests must pass test and CodeQL checks.
Deployment	The public App Service URL responds after deployment.

2. Mock UI: Dashboard

Screen purpose Let the user compare supported cities quickly, then select one for more information.

● ● ● Weather & Map

Weather around the world

Select a city to see its map and weather details

AU AUSTRALIA



Sydney

24°C

[View details ›](#)

SG SINGAPORE



Singapore

30°C

[View details ›](#)

IN INDIA



Bengaluru

27°C

[View details ›](#)



Mumbai

29°C

[View details ›](#)

Dashboard requirements

- Responsive Bootstrap 5 layout for mobile, tablet, and desktop.
- Country is shown with its full name and national flag, not an acronym.
- Every card shows city, current temperature in °C, and one weather emoji.
- Loading, empty, and friendly error states are visible.
- Cards are keyboard accessible and show a clear focus state.

3. Mock UI: City detail

Screen purpose Show location and richer weather information without making the experience complex.

[◀ Back to dashboard](#)

sg Singapore

MAP

● Singapore

[zoom +] [zoom -]

☁ 30°C

Light rain

Feels like 33°C

Min / Max 27° / 31°

Humidity 78%

Local time 4:30 PM

Detail requirements

- Map is centered on the selected city and displays one location marker.
- Weather panel shows condition, current temperature, minimum/maximum temperature, and humidity.
- Back returns to the dashboard without losing the page state.
- Unknown or unsupported city input returns a clear message rather than a broken screen.

Simple states

Loading	Success	Error
Spinner + “Loading weather...”	Map and weather panel shown	“Unable to load data. Try again.”

4. Backend and data flow

Recommended shape One small Node.js and Express service serves the frontend and protects the Azure Maps key.



Backend responsibilities

- Serve the built frontend as static files.
- Store supported city and country reference data in a single server-side configuration module.
- Resolve a supported city to coordinates through Azure Maps Search or use approved stored coordinates.
- Request current weather from Azure Maps using the server-side key.
- Validate city and country values before making an external request.
- Normalize the Azure Maps response into a small, stable JSON response for the UI.
- Return suitable HTTP status codes and safe error messages without exposing keys or raw provider errors.

Required endpoints

Endpoint	Purpose	Result
GET /api/health	Deployment health check	200 + status
GET /api/cities	Supported country/city data	Flags, coordinates, time zones
GET /api/weather?city=&country=	Current conditions for one city	Normalized weather JSON

Configuration

Required environment value `AZURE_MAPS_KEY`. The service must stop at startup with a clear error if it is missing. Provide an `.env.example` file with the name only, never a real key.

Example backend response

```
{ city: "Singapore", country: "Singapore", temperatureC: 30, condition: "Light rain", emoji: "☔", humidity: 78, latitude: 1.3521, longitude: 103.8198 }
```

5. Quality, security, and delivery

Testing

- Backend: response parsing, supported input, missing input, and provider failure.
- Frontend: dashboard rendering, card data binding, and click navigation.
- All tests run locally with a single documented command.
- CI runs tests on push and pull request to main.

Security

- Keep AZURE_MAPS_KEY in local environment settings, GitHub Actions secrets, and the App Service application setting.
- Do not call Azure Maps directly from browser code.
- Use `textContent` or safe DOM creation for values returned by APIs.
- Enable Dependabot and CodeQL. Require the test and CodeQL checks before merge.
- Do not log secrets or return raw external-service errors to the browser.

Deployment

- Use a multi-stage Dockerfile that packages the backend and built frontend.
- Publish the image to GitHub Container Registry with commit SHA and latest tags.
- Deploy the container to Azure App Service only after tests and security checks succeed.
- Authenticate GitHub Actions to Azure through OpenID Connect and a federated credential, without a client secret or publish profile.
- Verify deployment using `GET /api/health` and the application home page.

Definition of done

Check	Done when
☐ UI	Dashboard and detail view match the behaviors in this document.
☐ Backend	Endpoints validate input, protect the key, and return normalized data.
☐ Tests	Local and CI tests pass.
☐ Security	Dependabot and CodeQL are enabled; no secret is committed.
☐ Release	Container runs successfully in Azure App Service.

References

- Microsoft Learn: Azure Maps Weather service and current conditions REST API.
- Microsoft Learn: Authenticate to Azure from GitHub Actions by OpenID Connect.
- Internal training transcripts and prior GitHub review feedback for the weather dashboard pattern.